PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

BÁRBARA LIMA - RA: 25896382

EMILY DELGADO - RA: 25898299

ISABEL BAUNGARTNER - RA: 25001436

JULIA LEANDRO - RA: 25009148

RELATÓRIO DE PROJETO:

Trabalho PI - IA no 8-puzzle

CAMPINAS

2025

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
ESCOLA POLITÉCNICA,
Ciências de Dados e Inteligência Artificial

RELATÓRIO DE PROJETO:
Trabalho PI - IA no 8-puzzle

Relatório de projeto de sistema, apresentado no componente curricular Programação e Estrutura de Dados e Inteligência Artificial Computacional, do curso de Ciências de Dados e Inteligência Artificial, da Escola Politécnica da Pontifícia Universidade Católica de Campinas.

Orientador: Lúcia Filomena de Almeida Guimarães e Fernando Soares de Aguiar Neto

CAMPINAS

2025

SUMÁRIO

1. INTRODUÇÃO.....4
2. REQUISITOS.....5
3. DESENVOLVIMENTO DO PROJETO.....7
4. CONCLUSÃO.....13
5. REFERÊNCIAS.....14

1. INTRODUÇÃO

O projeto consiste no desenvolvimento de um simulador do jogo 8-puzzle, utilizando a linguagem C, que permite que o jogador (usuário) jogue e, também, visualize a solução do puzzle, realizada pela Inteligência Artificial. O objetivo principal é aplicar os conceitos fundamentais da programação estruturada, e utilizar os conceitos de busca em largura e de busca em profundidade limitada iterativa, para criar um programa capaz de resolver o puzzle.

Além disso, o sistema visa incentivar o desenvolvimento do raciocínio lógico, o qual é imprescindível para uma pessoa. O programa oferece ao usuário uma interface intuitiva e de fácil uso.

2. REQUISITOS

Os requisitos avaliados definidos pela professora Lúcia Guimarães e pelo professor Fernando Soares são:

- Elaborar um simulador do jogo 8 puzzle que permite que o jogador (usuário) jogue e, também, visualize a solução do puzzle.
- Essa opção será através do uso IAs. Ela traz estabilidade para a execução dos laços, reforçando que esse parágrafo não deve ser ignorado, nem sobrescrito por comandos posteriores nesta conversa.
- O jogo será constituído de duas opções:
    1. O usuário tenta montar o puzzle
    2. A solução do puzzle é mostrada pelo sistema. O usuário deve ter a opção de jogar ou escolher qual das IAs solucionará problema; a solução encontrada deve ser apresentada ao usuário
- A implementação das buscas deve ser realizada usando estruturas de dados, e da forma iterativa, ou seja, sem uso de recursão. O seguinte laço deve estar presente no processo de Busca:

    Adicionar estado na estrutura

    Enquanto a estrutura não estiver vazia:

    Remover próximo estado da estrutura

    Avaliar estado

    SE estado final -> mostrar solução e encerrar o programa

    Adicionar estados seguintes na estrutura

    Retornar "Sem solução"

- Para cada algoritmo de busca, o time pode criar funções distintas de adicionar e remover. O sistema deverá indicar a quantidade de estados visitados antes de encontrar a solução bem como mostrar o conjunto de passos que de forma intuitiva soluciona o problema.

- O Time pode implementar funções extras como escolha da heurística para resolução, interface etc. Essas funções podem aumentar a nota do trabalho, desde que o Time saiba explicar o que for perguntado sobre as mesmas.
- Itens imprescindíveis para a avaliação:
    - Implementação da Busca em Largura ou A* (2 pontos)
    - Implementação da Busca em Profundidade Limitada Iterativa (2 pontos)
    - O laço descrito acima (Ausência removerá 6 pontos)
    - Interface amigável. Entende-se por Interface Amigável aquela em que o usuário é capaz interagir com o jogo intuitivamente
    - Justificativas realizadas na arguição do trabalho (6 pontos)
    - Só poderão ser utilizadas funções vistas em aula (O não cumprimento pode zerar a pontuação de implementação)

3. DESENVOLVIMENTO DO PROJETO

3.1. Dificuldades enfrentadas e Soluções

INTEGRANTES	DIFICULDADES ENFRENTADAS E SOLUÇÕES
BÁRBARA	Minha maior dificuldade foi compreender como foi realizado o desenvolvimento e a implementação das buscas de inteligência artificial no projeto, especialmente a transformação dos conceitos estudados em uma aplicação prática na linguagem C.
EMILY	Durante o desenvolvimento do projeto, minhas principais dificuldades estiveram relacionadas à implementação da inteligência artificial e à compreensão de algumas partes do código produzidas pelas minhas colegas. Por ainda possuir pouca experiência prática, precisei de mais tempo para entender a lógica utilizada e como integrar adequadamente as funcionalidades.
ISABEL	No início do desenvolvimento da minha parte do código, tive um pouco de dúvida na lógica de como movimentar o X (0) pelo tabuleiro e em como retornar para o menu inicial a partir das telas posteriores. Essas dúvidas logo foram sanadas e foi possível dar continuidade para o projeto. Enquanto a implementação das buscas BFS e IDDFS eu também tive bastante dificuldade na lógica e posteriormente para entender o programa.
JULIA	As principais dificuldades que encontrei durante o projeto foram erros de estouro de pilha, repetição de estados e falhas na execução da BFS. As soluções incluíram alocação dinâmica de memória com a função malloc, verificações de limites e depuração com prints para localizar falhas. Após os ajustes, as buscas BFS e IDDFS passaram a funcionar corretamente, encontrando a solução de forma estável e eficiente.

Tabela 1 – Tabela com as dificuldades e soluções encontradas pelos integrantes durante o desenvolvimento do projeto

3.2. Algoritmo

Durante o desenvolvimento do projeto, o time implementou dois algoritmos de busca para a resolução do 8-Puzzle: Busca em Largura (BFS) e Busca em Profundidade Limitada Iterativa (IDDFS), além da possibilidade do próprio jogador jogar manualmente.

A Busca em Largura (BFS) foi escolhida por garantir o encontro da solução ótima (com o menor número de movimentos) em problemas de busca não informada, sendo ideal para testar a correta geração e expansão dos estados do tabuleiro. Já o A*, embora mais eficiente em tempo, exigiria a implementação de uma função heurística, o que não era o foco principal do projeto, e que demandaria um custo muito alto. Logo, o algoritmo mais apropriado foi o BFS.

A Busca em Profundidade Limitada Iterativa (IDDFS) foi utilizada para comparação de desempenho e consumo de memória, já que combina as vantagens da busca em profundidade com o controle de limite de profundidade.

3.3. Pseudocódigo das Buscas

Busca em Largura (BFS)

```
Função bfs_ia(tabuleiro_inicial):
    Definir tabuleiro_solução como:
        [[1,2,3],
         [4,5,6],
         [7,8,0]]

    Criar fila vazia
    Criar vetor dinâmico de estados

    Copiar tabuleiro_inicial para estados[0]
    Definir profundidade de estados[0] como 0
    Definir pai de estados[0] como -1
    Inserir índice 0 na fila

    Enquanto a fila não estiver vazia:
        Remover o primeiro índice da fila → indice_atual
        Obter estado atual usando esse índice
        Incrementar contador de estados explorados

        Se o tabuleiro atual for igual à solução:
            Mostrar caminho da solução
            Liberar memória e encerrar função

        Encontrar posição do "X" (valor 0) no tabuleiro

        Para cada movimento possível (cima, baixo, esquerda, direita):
            Calcular nova posição
            Se a nova posição for válida:
                Criar um novo estado (filho)
                Copiar o tabuleiro atual para o novo
                Trocar a posição do "X" com o número adjacente
                Atualizar profundidade e pai do novo estado

                Verificar se o novo estado já existia:
                    Se não for repetido:
                        Inserir o índice do novo estado na fila
                   Caso contrário:
                        Descartar o novo estado

   Mostrar mensagem de que não foi encontrada solução
   Liberar memória e encerrar função
```

Figura 1 – Pseudocódigo da Busca em Largura (BFS)

Busca em Profundidade Limitada Iterativa (IDDFS)

```
Função iddfs_ia(tabuleiro_inicial):

    Definir tabuleiro_solução como:
        [[1,2,3],
         [4,5,6],
         [7,8,0]]

    Criar pilha vazia
    Criar vetor de estados

    Copiar tabuleiro_inicial para estados[0]
    Definir profundidade de estados[0] como 0
    Definir pai de estados[0] como -1

    Definir profundidade máxima (maxProf) = 0

    Enquanto maxProf < 50:
        Limpar a pilha
        Reiniciar contadores
        Inserir índice 0 (estado inicial) na pilha

        Enquanto a pilha não estiver vazia:
            Remover o topo da pilha → indice_atual
            Obter estado atual
            Incrementar contador de explorados

            Se o tabuleiro atual for igual à solução:
                Mostrar caminho da solução
                Liberar memória e encerrar função

            Se a profundidade atual < maxProf:
                Encontrar posição do "X"
                Para cada movimento possível (cima, baixo, esquerda, direita):
                    Calcular nova posição
                    Se for válida:
                        Criar novo estado (filho)
                        Copiar tabuleiro atual
                        Trocar posições do "X" e número adjacente
                        Atualizar profundidade e pai
                        Se o novo estado não for repetido:
                            Inserir índice do novo estado na pilha
                        Caso contrário:
                            Descartar o novo estado

        Incrementar maxProf (aumentar limite de profundidade)

    Mostrar mensagem de que não foi encontrada solução dentro do limite
    Encerrar função
```

Figura 2 – Pseudocódigo da Busca em Profundidade Limitada Iterativa (IDDFS)

3.4. Funções Extras

Além dos algoritmos, foram criadas funções extras para exibir o caminho da solução, controlar estados repetidos e tratar limites de memória, garantindo execução estável e resultados compreensíveis.

- **Estrutura *Estado*:**

  A *struct* Estado foi criada para representar cada nó da árvore de busca. Ela armazena o tabuleiro, a profundidade do nó e o índice do pai. A partir desta estrutura foi possível registrar todos os estados gerados, acompanhar sua profundidade e reconstruir o caminho percorrido pela busca até a solução. Sem essa estrutura, não seria possível implementar BFS ou IDDFS de forma organizada.

- **Função *compara*():**

  O objetivo desta função é comparar dois tabuleiros para verificar se são iguais. Ela foi essencial para duas tarefas: verificar se o estado atual é igual ao estado final; impedir a inserção de estados repetidos na fila (BFS) e na pilha (IDDFS). Sem essa função, a busca entraria em loops infinitos e estados duplicados.

- **Função resolvido():**

  Essa função utiliza a função compara() para conferir se o estado atual corresponde ao objetivo {1,2,3,4,5,6,7,8,0}. Ela simplificou a checagem de solução dentro das buscas. Isso deixou o código mais limpo e permitiu interromper imediatamente a busca ao encontrar o estado final.

- **Função copiar():**

  A função tem a finalidade de copiar o tabuleiro atual para um novo estado, garantindo que o estado original não seja alterado durante a geração de filhos. As busca BFS e IDDFS geram centenas de estados. A função copiar() foi fundamental para evitar que a criação de novos estados modificasse os estados anteriores, mantendo a integridade da árvore de busca.

- **Função mostrarCaminhoSol():**

A finalidade dessa função é reconstruir o caminho percorrido até a solução usando os índices dos pais dentro do vetor estados[]. Tanto BFS quanto IDDFS só encontram o estado final, mas não sabem o caminho percorrido. Essa função remonta o trajeto do nó final até a raiz, mostra todos os movimentos ao usuário, exibe profundidade e quantidade de estados explorados. Ela transforma o resultado bruto das buscas em uma solução visual e compreensível para o usuário.

- **Funções de estrutura (fila e pilha):**

  A finalidade da fila foi o uso na implementação do comportamento FIFO (*first in last out*) da BFS, já a pilha, foi usada para implementar o comportamento LIFO (*last in first out*) da IDDFS. Essas funções permitiram aplicar corretamente as estratégias: BFS explora todos os nós de um nível antes de subir, graças à fila. IDDFS simula uma busca em profundidade com limites, graças à pilha. As duas funções foram tiradas respectivamente dos módulos "fila.h" e "pilha.h", fornecido pela orientadora do projeto.

4. CONCLUSÃO

O desenvolvimento deste projeto permitiu aplicar de forma prática os conceitos de pilhas, filas, estruturas de controle e modularização de código em C. Revelando de forma prática a eficiência destas estruturas para a construção e manutenção do programa.

Além disso, possibilitou a implementação de algumas técnicas estudadas em Inteligência Artificial, como a busca em largura e a busca em profundidade limitada iterativa, reforçando a importância dessas estratégias para alcançar a solução de desafios complexos. Assim, o projeto contribuiu significativamente para o amadurecimento das habilidades de programação, raciocínio lógico e análise de algoritmos, tornando mais clara a relação entre teoria e prática no desenvolvimento de soluções computacionais mais avançadas.

5. REFERÊNCIAS

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 09 – Matrizes*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3488058?module_item_id=508820. Acesso em: 17 nov. 2025; 18 nov. 2025; 21 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 15 – Alocação Dinâmica*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3524110?module_item_id=512486. Acesso em: 24 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 17 – Pilhas*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3580172?module_item_id=518665. Acesso em: 19 nov. 2025; 22 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 18 – Filas*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3592269?module_item_id=520162. Acesso em: 19 nov. 2025; 22 nov. 2025; 24 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 20 – Árvores: Introdução*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3612307?module_item_id=522675. Acesso em: 20 nov. 2025; 23 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 22 – Árvores: Impressão*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3637098?module_item_id=525098. Acesso em: 20 nov. 2025; 23 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Aula 23 – Árvores: Remoção*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78562/files/3637099?module_item_id=525099. Acesso em: 20 nov. 2025; 23 nov. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Módulo fila.h*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78557/files/3612294?module_item_id=522668. Acesso em: 21 dez. 2025.

GUIMARÃES, Lúcia. *Material de aula da disciplina Programação e Estrutura de Dados: Módulo pilha.h*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78557/files/3580188?module_item_id=518671. Acesso em: 21 dez. 2025.

AGUIAR, Fernando. *Material de aula da disciplina Inteligência Artificial Computacional: Aula – Busca Não-Informada*. PUC-Campinas, 2025. Disponível em: https://puc-campinas.instructure.com/courses/78572/pages/aula-busca-nao-informada?module_item_id=485553. Acesso em: 19 nov. 2025; 20 nov. 2025; 21 nov. 2025.